



Report

bachelors of Computer Application

Semester – V

Sub:

Deep Learning

Name: Bellary Dev Nidhi

Reg no.: 2411021240065

Faculty Name: Nagasri

Faculty Signature:

Department of Computer Application

Alliance University

Chandapura - Anekal Main Road, Anekal Bengaluru - 562 106

Comparison of Deep Learning Architectures

Feedforward Networks, Convolutional Neural Networks, and Recurrent Neural Networks

Introduction

Deep learning is not one technique but a family of architectures, and each family was built to deal with a particular kind of data. This project looks at three of the foundational ones — the Feedforward Neural Network (FNN), the Convolutional Neural Network (CNN), and the Recurrent Neural Network (RNN) — and works through how each is structured, what kind of input it expects, and where it actually gets used in practice. Rather than treating these as interchangeable “neural networks,” the goal here is to build a clear sense of why a particular architecture is chosen for a particular problem: why nobody feeds raw images into a plain feedforward net, and why an RNN and not a CNN is the natural fit for a sentence.

Task 1: Basic Architecture of Each Network

1.1 Feedforward Neural Network (FNN)

The feedforward network is the most basic deep learning architecture, and it is also the one every other architecture eventually builds on. It consists of an input layer, one or more hidden layers, and an output layer. Every neuron in one layer connects to every neuron in the next (this is why it is often called a “densely connected” or “fully connected” network), and the connections never loop backward — information moves strictly forward, from input to output, which is exactly what the name describes. Each connection carries a weight, each neuron applies an activation function such as ReLU or sigmoid, and training happens through backpropagation, where the error at the output is propagated back through the layers to adjust the weights. Because there are no loops and no memory of past inputs, an FNN treats every input example as completely independent of the ones before it.

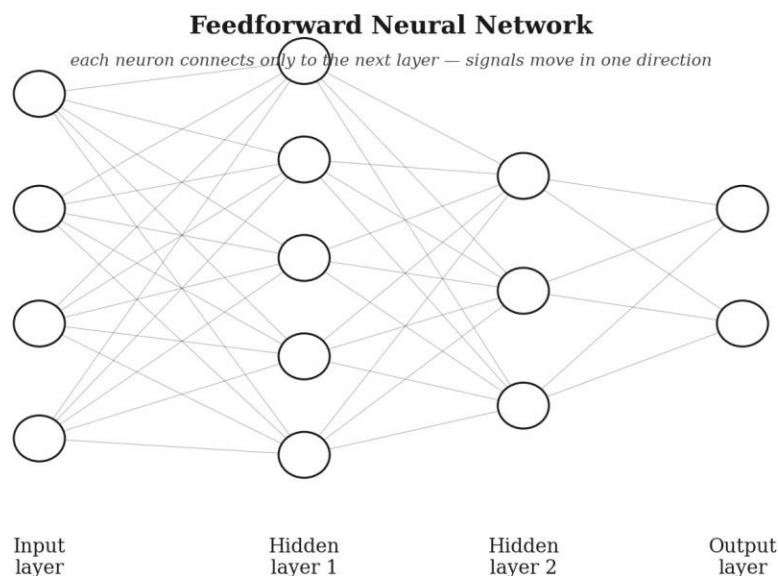


Fig. 1 — A feedforward network: information passes forward through fully connected layers, with no loops.

1.2 Convolutional Neural Network (CNN)

CNNs were built specifically to deal with data that has spatial structure — images, most obviously. Instead of connecting every neuron to every pixel (which would be computationally enormous and would also throw away the spatial relationships between neighbouring pixels), a CNN uses small learnable filters (kernels) that slide across the input and compute a convolution at each position. This produces feature maps that pick up on local patterns — edges, corners, textures in the early layers, and increasingly abstract shapes in deeper layers. Pooling layers (typically max-pooling) then downsample these feature maps, reducing their size while keeping the most important activations, which makes the network more efficient and a bit more tolerant to small shifts in the image. After several rounds of convolution and pooling, the resulting feature maps are flattened and passed into fully connected layers, which behave exactly like a standard FNN and produce the final classification or prediction. The two properties that make CNNs efficient are local connectivity (a neuron only looks at a small patch of the input, not the whole thing) and parameter sharing (the same filter is reused across the entire image rather than learning a separate weight for every pixel position).

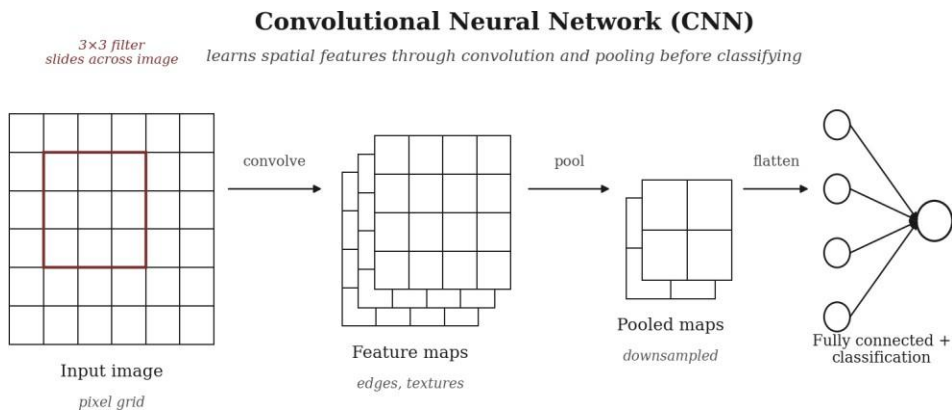


Fig. 2 — A CNN pipeline: convolution extracts features, pooling downsamples, and dense layers classify.

1.3 Recurrent Neural Network (RNN)

RNNs exist to handle sequential data, where the order of inputs matters and where earlier inputs should influence how later ones are interpreted — something neither an FNN nor a CNN is built to do. An RNN processes a sequence one element at a time, and at each step it maintains a hidden state that is updated using both the current input and the hidden state from the previous step. This hidden state effectively acts as the network's memory, letting information from earlier in the sequence carry forward. If you “unroll” an RNN across time steps, it looks like a chain of repeated cells, all sharing the same set of weights, each passing its hidden state to the next. This is what lets an RNN handle inputs of variable length and makes it naturally suited to text, audio, and any data where context builds up over time. In practice, plain RNNs struggle with long sequences because gradients tend to vanish (or explode) as they are propagated back through many time steps, which is why variants like LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) are used far more often in real applications — they add gating mechanisms that control what gets remembered and what gets forgotten.

Recurrent Neural Network (RNN)

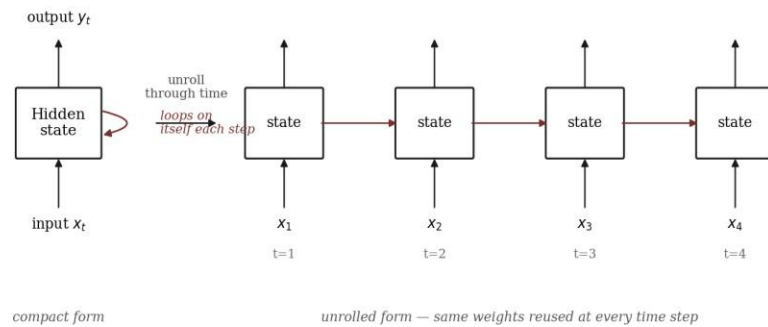


Fig. 3 — An RNN cell shown in compact form (left) and unrolled across time steps (right).

Task 2: Suitable Applications for Each Architecture

The applications below follow directly from the structural differences covered in Task 1 — an architecture's strengths are really a description of what kind of data it was designed to preserve information about.

Feedforward Neural Networks

- Tabular data problems — predicting house prices, credit scoring, customer churn prediction, where inputs are a fixed set of numeric/categorical features.
- Basic pattern recognition tasks on structured/pre-processed data, e.g. simple digit or handwriting classification on flattened pixel vectors.
- As the final classification stage inside larger architectures — both CNNs and RNNs typically end with fully connected (feedforward) layers.
- Simple recommendation and scoring systems where the order of features doesn't matter.

Convolutional Neural Networks

- Image classification — identifying objects, animals, handwritten digits (e.g. the classic MNIST task).
- Object detection and localisation, used in autonomous vehicles and surveillance systems.
- Medical imaging — detecting tumours or abnormalities in X-rays, MRIs, and CT scans.
- Facial recognition and biometric authentication.
- Video analysis, where CNNs are applied frame-by-frame or combined with recurrent layers for motion.

Recurrent Neural Networks

- Natural language processing — language modelling, machine translation, text generation.
- Speech recognition, where audio is inherently sequential.
- Time-series forecasting — stock price prediction, weather forecasting, sensor/IoT data analysis.
- Sentiment analysis and chatbots, where meaning depends on word order and context.

- Music generation and other sequence-to-sequence generation tasks.

Task 3: Comparative Analysis

Aspect	Feedforward Neural Network	CNN	RNN
Core structure	Layers of neurons fully connected in sequence, no loops or memory of previous inputs	Convolutional layers (filters) + pooling layers, followed by fully connected layers	Neurons with a hidden state that feeds back into itself at every time step
Data flow	One direction only — input to output, no cycles	One direction, but filters slide spatially across the input	Sequential, with the hidden state carried forward across steps
Ideal input	Fixed-size numeric/tabular vectors	Grid-like data — images, spectrograms	Sequential data — text, time series, audio
Handles order/context?	No — treats each input independently	Partially — captures local spatial patterns, not sequence order	Yes — designed specifically to model order and context
Parameter sharing	None — every connection has its own weight	Yes — the same filter is reused across the image	Yes — the same weights are reused at every time step
Typical use cases	Classification/regression on tabular data, simple pattern recognition	Image classification, object detection, medical imaging	Language modelling, speech recognition, stock/time-series prediction
Key limitation	Cannot handle spatial or sequential structure well; needs a fixed input size	Not naturally suited to sequential or ordered data	Struggles with very long sequences (vanishing gradients) unless using LSTM/GRU variants

Conclusion

The three architectures are best understood as answers to three different questions about the input data. Feedforward networks answer the question of independent, fixed-size inputs; CNNs answer the question of spatial structure and locality, which is why images are their natural domain; and RNNs answer the question of order and context over time, which is why they dominate language and sequence tasks. In practice, real-world systems often combine these ideas — for instance, a CNN might extract features from video frames while an RNN models how those features change over time — but understanding each architecture on its own is what makes it possible to reason about why such combinations work.